Reading: code coverage

Objectives

- Read a JaCoCo report for line and branch coverage
- Explain why high coverage does not prove code is correct
- Find an untested branch and add a test for it

Teaching

Activity 1 of 3: Predict how a coverage report marks a line

Before any explanation: a coverage tool records which lines and decisions your tests execute. Here is a class and its only test. Predict how the tool marks the line `if (orderTotal >= 50) {` after the test runs.

Shipping.java

```
class Shipping {
    int cost(int orderTotal, boolean express) {
        int cost = 5;
        if (orderTotal >= 50) {
            cost = 0;
        }
        if (express) {
            cost = cost + 10;
        }
        return cost;
    }
}
```

The only test

```
@Test
void smallStandardOrderPaysBaseCost() {
    assertEquals(5, new Shipping().cost(20, false));
}
```

Activity 2 of 3: Find the untested branches and test them

JaCoCo is a coverage tool for Java. During a test run it records what executed, then its HTML report colors each source line: green when all of the line's instructions ran, yellow when only some did, and red when none did.

Lines with an `if` or `switch` also get a diamond for branch coverage: green when every branch was taken, yellow when some were, red when none were. A hint such as "1 of 2 branches missed" tells you which test is still to write.

To raise branch coverage, add a test whose inputs send a decision the other way, and still assert the expected result.

Activity 3 of 3: Explain what coverage does not prove

Coverage tells you what ran, not whether the result was right. A test that calls a method and checks nothing still executes every line it reaches, so the report turns green.

A wrong expected value is just as invisible: coverage counts the line the same way whether the assertion is correct, wrong or missing. Use coverage to find code no test reaches; use assertions, and tests that fail when you break the code on purpose, to judge correctness.

Recap

- JaCoCo marks lines green (all instructions ran), yellow (some ran) or red (none ran).
- Branch coverage counts the outcomes of each `if` or `switch`; "1 of 2 branches missed" means one outcome was never taken.
- To cover a missed branch, add a test whose inputs take that branch, and assert the expected result.
- High coverage shows code ran, not that it is right: a test with no assertion, or a wrong one, covers lines just as well.

Cheat sheet

Where to act: The test's call, `cost(20, false)`, and the first `if`.

Next step: Decide whether the condition is true or false for 20, and whether the other outcome ever happens in this test run.

Where to act: The body of `largeOrderShipsFree`, aimed at the first partly covered `if` line in the coverage view.

Next step: Its missing side is the true branch: choose an order total that makes `orderTotal >= 50` true and work out the cost by hand before you assert it.

Another hint Another hint, 2 of 2 for Activity 2

Where to act: The body of `expressOrderAddsTen`, aimed at the second `if`, on `express`.

Next step: Every call so far passes `false`; in this test pass `true`, and decide what cost that order should have when you assert it.

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class ShippingTest {
    @Test
    void smallStandardOrderPaysBaseCost() {
        assertEquals(5, new Shipping().cost(20, false));
    }

    @Test
    void largeOrderShipsFree() {
        // Pick a total that takes the untested side
        // of the first if; assert the cost you expect.
    }

    @Test
    void expressOrderAddsTen() {
        // Pass true for express; assert the cost you expect.
    }
}
```

- It reads a small Java subset: classes and interfaces with fields, constructors and methods; `if`/`else`, for-each over a `List`, `return` and `throw`; `String`, `int`, `long`, `double`, `boolean`, `List` and `Map`; and lambdas.
- It knows JUnit Jupiter `@Test`, `@BeforeEach` and the common `Assertions` methods, plus Mockito `mock`, `when`/`thenReturn`, `verify`, `times`, `never` and a few argument matchers.
- Anything else shows Not supported here: that is a limit of this page, not a claim that the Java is wrong. Error is used only where javac or JUnit certainly rejects the code.
- It is not javac or the JUnit Platform: it runs tests in source order (JUnit itself does not promise an order), shortens failure messages, and does not perform every compile-time check. Confirm real

projects with Maven or your IDE.

Where to act: The body of `runsEveryLine`.

Next step: Look for the line that would fail if `cost` returned the wrong number; ask what happens when there is none.

```
// Report reading
// green line: every instruction ran
// yellow line: some instructions ran
// red line: nothing ran
// diamond on if/switch: branches taken
//   "1 of 2 branches missed"

@Test
void memberPriceTakesTheDiscountBranch() {
    // inputs chosen to take a missed branch,
    // with the expected result worked out by hand
    assertEquals(90, new Pricing().price(100, true));
}
```

Claims on this page were checked against these official sources, retrieved 2026-10-16. The practice coverage view marks statement lines and if-branches from the runner; JaCoCo measures bytecode instructions and branches, so a real report can also mark lines such as a method’s closing brace.

- JaCoCo: Coverage Counters — line states (green, yellow, red), branch counters for if and switch, and the diamond markers
- JaCoCo documentation — what JaCoCo is and how reports are produced
- JaCoCo Maven plug-in — running coverage in a Maven build
- JUnit User Guide: Assertions — assertEquals with the expected value first

Static interaction alternatives

predict

Question: How is that if line marked?. Option A: Fully covered: the line ran - Not quite. The line ran, but an if has two outcomes and only one of them happened. Option B: Partly covered: the line ran, but only one of its two branches was taken - Correct. With 20 the condition is false every time, so the true branch, cost = 0;, never runs. A line whose decision went only one way is partly covered. Option C: Not covered: no test reached it - Not quite. Every call to cost evaluates this condition, so the line is reached. Option D:

Not marked at all, because coverage tools ignore if lines - Not quite. Decision lines are exactly where branch coverage is measured. Before feedback, the learner rates confidence: Guessing, Fairly sure or Certain.

explore

Practice JUnit editor with a coverage view. Shipping.cost(orderTotal, express) starts at 5, sets the cost to 0 when orderTotal is 50 or more, then adds 10 when express is true. ShippingTest.java has one test, cost(20, false) equals 5, which leaves the line cost = 0 and the line cost = cost + 10 not covered and each if partly covered (one of two branches). It also has two empty test methods, largeOrderShipsFree and expressOrderAddsTen. Task: fill in those two tests with assertions whose inputs take the missing branches until every line runs and both ifs show 2 of 2 branches. Checks: both named tests assert a cost, pass, and fail when the branch each tests is broken on purpose; all branches taken, all lines executed, all tests pass and assert a result.

mcq

Question: What does runsEveryLine prove?. Option A: That cost returns the right value for every input, because coverage is 100% - Not quite. Coverage only shows that the lines ran; nothing compares a result with an expected value. Option B: Only that both calls finish without throwing; it would still pass if cost returned wrong numbers - Correct. With no assertion, the test passes whatever the method returns, so full coverage proves nothing about correctness. Option C: Nothing, because JUnit fails any test that has no assertion - Not quite. JUnit passes a test that finishes without an exception, with or without assertions. Option D: That the coverage tool reports 0%, because coverage counts only assertions - Not quite. Coverage counts executed code, and both branches of both ifs run here. Before feedback, the learner rates confidence: Guessing, Fairly sure or Certain.